

Object Design Document (ODD)

MotorsRent

Riferimento	
Versione	1.0
Data	24/11/2025
Destinatario	Azienda ElitéMotors S.r.l.
Presentato da	Ingenito Davide Sorrentino Luca Teodonno Francesco
Approvato da	

Revision History

Data	Versione	Descrizione	Autori
24/11/2025	1.0	OD Trade-off	Team
24/11/2025	1.0	Linee Guida per la Documentazione delle Interfacce	Team
24/11/2025	1.0	Design Pattern	Team
24/11/2025	1.0	Definizioni, acronimi e abbreviazioni	Team

Revision History	2
1 Introduzione	4
1.1 Object Design Trade-offs	5
1.2 Linee Guida per la Documentazione delle Interfacce	5
1.2.1 Classi e interfacce Java.....	5
1.2.2 Commenti Javadoc	5
1.2.3 Pagine HTML	6
1.2.4 File Javascript	6
1.2.5 Naming Conventions Generali	6
1.2.6 Regole di Indentazione e Formattazione	6
1.3 Desin Pattern.....	7
1.4 Definizioni, acronimi ed abbreviazioni	9

1. Introduzione

1.1 Object Design Trade-offs

Di seguito sono elencati principali compromessi architetturali e motivazioni:

Separazione responsabilità vs. semplicità

- *Scelta:* adozione del pattern MVC con Service Layer e DAO.
- *Motivazione:* migliora manutenibilità e testabilità, ma introduce maggiore complessità iniziale e più classi da implementare.

Performance vs. consistenza immediata

- *Scelta:* uso di cache (Cache-Aside) per il catalogo e letture frequenti;
- *Motivazione:* riduce latenza (RNF_P_1) e supporta carico concorrente (RNF_P_2) a scapito di eventuale consistenza eventuale nelle viste derivate (statistiche) che verranno trattate come Eventual Consistency e documentate.

Singleton (DbConnection) vs. Dependency Injection

- *Scelta:* implementare un singleton per centralizzare la gestione della DataSource, ma favorire l'uso di DI nei componenti per facilitare test e mock.
- *Motivazione:* singleton semplifica la gestione della connessione in esercitazioni e piccoli deployment; DI aumenta testabilità e flessibilità in ambienti enterprise.

Gestione notifiche: Observer vs. chiamate sincrone dirette

- *Scelta:* Observer + Event-Driven per notifiche e invio email (asynchronous), con fallback sincro quando necessario.
- *Motivazione:* migliora reattività dell'UI e resilienza del sistema, ma richiede infrastruttura di messaging.

Modello dati minimalista vs. modellazione completa delle vendite

- *Scelta:* mantenere le quattro entità principali (Utente, Auto, Preventivo, Leasing) e derivare statistiche dalle richieste; non creare entità Vendita per non complicare il data model iniziale.
- *Motivazione:* semplicità e velocità di sviluppo; possibile estensione futura aggiungendo l'entità Vendita se richiesto.

1.2 Linee Guida per la Documentazione delle Interfacce

Le seguenti linee guida stabiliscono gli standard obbligatori per la scrittura, la formattazione e la documentazione delle componenti software del sistema MotorsRent. Tali linee guida seguono un approccio caratterizzato da prescrizioni chiare e uniformi, al fine di garantire coerenza, leggibilità e manutenibilità del codice prodotto da tutto il team.

1.2.1 Classi e Interfacce Java

Le classi e le interfacce Java devono essere implementate rispettando il seguente insieme di regole formali:

Norme:

- Le parentesi graffe di apertura devono essere posizionate **sulla stessa riga** della dichiarazione.
- Ogni classe deve contenere una **intestazione Javadoc** descrittiva.
- I nomi delle classi devono utilizzare **PascalCase** (es. CarRequestManager).
- È obbligatorio documentare eccezioni, parametri e valore di ritorno.

1.2.2 Commenti Javadoc

La documentazione Javadoc è obbligatoria per:

- Tutte le classi
- Tutti i metodi pubblici e protetti
- Tutte le interfacce

Norme:

- I commenti devono iniziare con una frase descrittiva sintetica.
- Devono includere i tag: `@param`, `@return`, `@throws` quando necessario.
- Il testo deve essere chiaro, conciso e conforme al linguaggio tecnico del progetto.

1.2.3 Pagine HTML

Le pagine HTML devono essere conformi agli standard **HTML5** e seguire precise regole di formattazione.

Norme:

- Ogni tag deve essere indentato di 2 spazi rispetto al contenitore.
- I tag devono essere chiusi correttamente.
- È vietato inserire più elementi inline nella stessa riga.
- Il codice deve essere semanticamente corretto (uso appropriato di header, nav, section, article, footer).

1.2.4 File JavaScript

Gli script JavaScript devono seguire il Google JavaScript Style Guide.

Norme:

- Indentazione: 2 spazi.
- Dichiarazioni obbligatorie con const e let, non usare var.
- Le funzioni devono essere nominate in camelCase.
- Obbligo di JSDoc per funzioni complesse o metodi pubblici.

1.2.5 Naming Conventions Generali

- I nomi devono essere descrittivi e privi di abbreviazioni non standard.
- I file devono avere lo stesso nome della classe o del componente che contengono.
- Le costanti devono essere scritte in MAIUSCOLO_CON_UNDERSCORE.

1.2.6 Regole di Indentazione e Formattazione

- Non sono ammessi trailing spaces.
- Ogni blocco logico deve essere separato da una riga vuota.
- Il limite massimo di linea è 120 caratteri.

1.3 Design Pattern

Per la realizzazione del sistema MotorsRent sono stati adottati alcuni design pattern che permettono di ottenere una struttura modulare, estendibile e facilmente manutenibile.

I pattern scelti risolvono problemi ricorrenti nella gestione della persistenza, del ciclo di vita degli oggetti e delle notifiche automatiche legate alle operazioni sulle entità principali del sistema.

I pattern utilizzati sono: **Singleton, DAO, Observer.**

Singleton Pattern

Il Singleton è un pattern che controlla il ciclo di vita di una classe garantendo che venga creata un'unica istanza condivisa, impedendo la generazione di istanze multiple non desiderate.

Nel progetto MotorsRent viene utilizzato per la gestione della connessione al database: ciò assicura un unico punto di accesso alle risorse di persistenza e riduce il rischio di creare più connessioni concorrenti.

DAO Pattern

Il DAO (Data Access Object) è un pattern architetturale che consente di separare la logica applicativa dalle operazioni di accesso ai dati.

Nel sistema *MotorsRent* ogni entità persistente (Utente, Auto, Preventivo, Leasing) possiede un corrispondente DAO che offre metodi dedicati per le operazioni CRUD (Create, Read, Update, Delete).

Questo approccio permette di:

- isolare la logica SQL dai Controller
- facilitare la manutenzione e la sostituzione del layer di persistenza
- mantenere il codice più leggibile e modulare

Facade Pattern

Il **Facade** è un design pattern strutturale che fornisce un'interfaccia unificata e semplificata per accedere a un insieme di interfacce in un sottosistema. Definisce un livello di astrazione più alto che rende il sottosistema più facile da utilizzare.

Nel progetto **MotorsRent**, il pattern è stato implementato attraverso la classe `MotorsRentFacade`. Questa classe funge da "cruscotto" centrale che si interpone tra i Controller (Servlet) e il livello di accesso ai dati (DAO e `DbConnection`).

L'adozione di questo pattern permette di:

- **Nascondere la complessità:** I Controller non devono più preoccuparsi di gestire l'apertura/chiusura delle connessioni al database o l'istanziamento dei singoli DAO.
- **Ridurre l'accoppiamento:** Le Servlet dipendono ora solo dalla classe Facade e non conoscono i dettagli implementativi delle classi di persistenza sottostanti.
- **Centralizzare la logica:** Le operazioni principali (es. `aggiungiAuto`, `getCatalogoAuto`) sono esposte come metodi semplici e diretti, migliorando la leggibilità e la manutenibilità del codice nei Controller.

1.4 Definizioni, acronimi e abbreviazioni

- **API:** Application Programming Interface
- **DAO:** Data Access Object
- **DTO:** Data Transfer Object
- **MVC:** Model–View–Controller
- **UI:** User Interface
- **DB:** Database
- **UI:** User Interface
- **RNF:** Requisito Non Funzionale
- **OPP:** Object-Oriented Programming
- **ID:** Identifier